Diabetes Hospital Readmission Prediction

Predicting 30-Day Readmission Risk Using Machine Learning

MSIN0097 Predictive Analytics — Individual Coursework

MSc Business Analytics | UCL School of Management

March 2026

Word count: 1995

Repository: https://github.com/annjalliii/Predictive-Analytics-Submission

1. Introduction and Problem Framing

Hospital readmissions pose a major challenge for healthcare systems worldwide. The US Hospital Readmissions Reduction Program (HRRP) enforces financial penalties on facilities with high 30-day readmission rates. Diabetic patients are especially vulnerable to readmission due to complex care needs, medication management, and co-morbidities. This project creates a binary classification system to predict whether a diabetic patient will be readmitted within 30 days of discharge, using encounter-level clinical data from 130 US hospitals.

The dataset used is the Diabetes 130-US Hospitals dataset from the UCI Machine Learning Repository, comprising 101,766 clinical encounters recorded between 1999 and 2008, with 50 features. The target variable was created by transforming the original three-class readmission label into a binary indicator: 1 if the patient was readmitted within 30 days, 0 otherwise.

Given the clinical context, AUC-ROC was chosen as the primary evaluation metric because of its robustness to class imbalance, supported by F1-score, PR-AUC, and recall. Accuracy was deliberately not used as a primary metric because of the severe class imbalance (89:11), which would make it misleading. Moreover, a naive majority-class classifier would reach about 89% accuracy. Cost asymmetry was also considered clinically, as missing a high-risk patient (false negative) has greater consequences than a false alert, making recall especially important.

2. Exploratory Data Analysis

After loading the raw data, an initial quality assessment uncovered several issues. Three columns had too many missing values: weight (97%), medical_specialty (49%), and payer_code (40%), leading to their removal. The placeholder character ‘?’ was used in seven columns instead of standard NaN values. Additionally, 1,652 records belonged to deceased patients, for whom readmission prediction is clinically irrelevant, so these were removed. After cleaning, the working dataset included 100,114 encounters across 28 features.

The target distribution exhibited significant class imbalance, as shown in Figure 1, with 88,757 non-readmitted encounters (88.7%) compared to 11,357 readmissions (11.3%), resulting in an imbalance ratio of 7.8:1. This highlights the need for specialised handling during both modeling and evaluation.

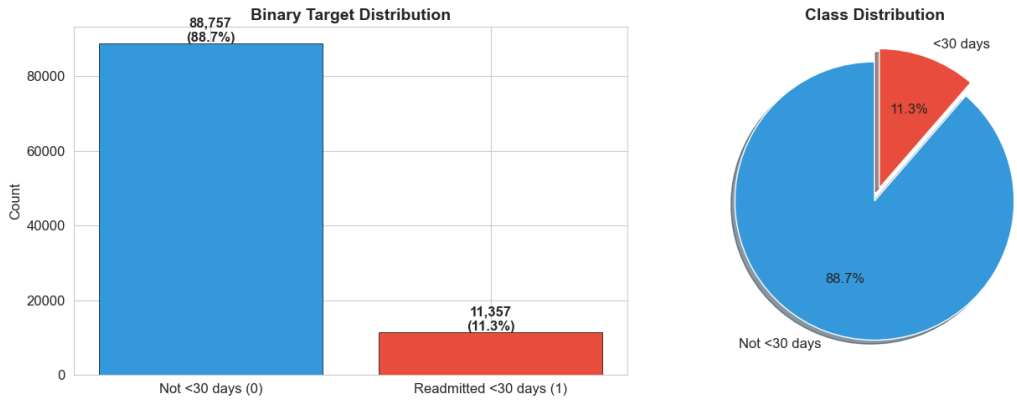

Figure 1: Distribution of binary target classes showing a 7.8:1 imbalance.

The correlation heatmap (Figure 2) showed that individual feature correlations with readmission were consistently weak (all below 0.17), with number_inpatient showing the strongest linear relationship (+0.168). This suggested that non-linear models capable of capturing feature interactions are likely to perform better than linear models.

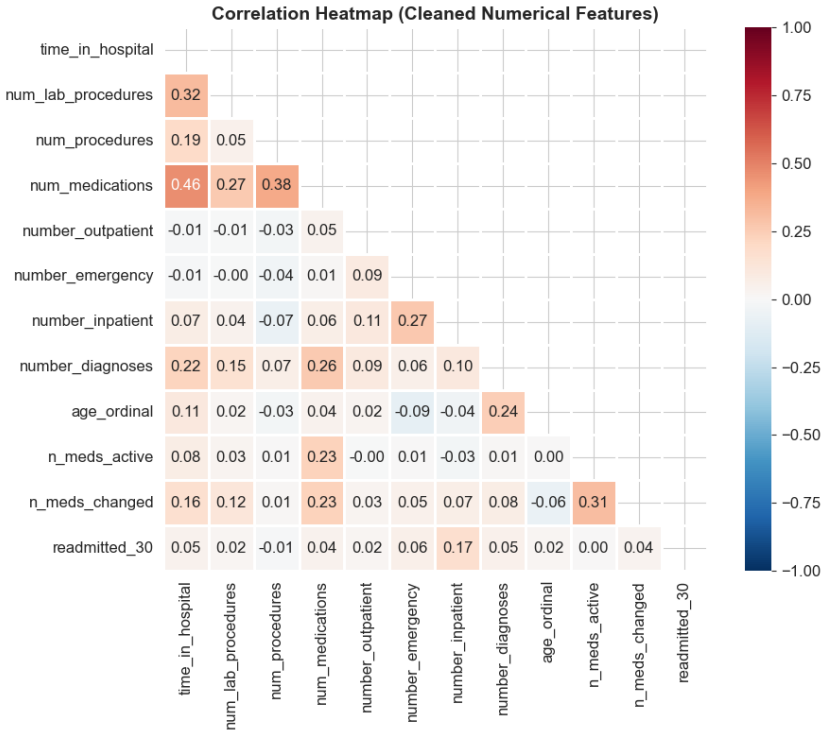

Figure 2: Heatmap showing the correlation among cleaned numerical features

Analysis of categorical features (Figure 3) revealed differences in readmission rates across diagnosis groups, insulin dosage changes, and discharge dispositions.

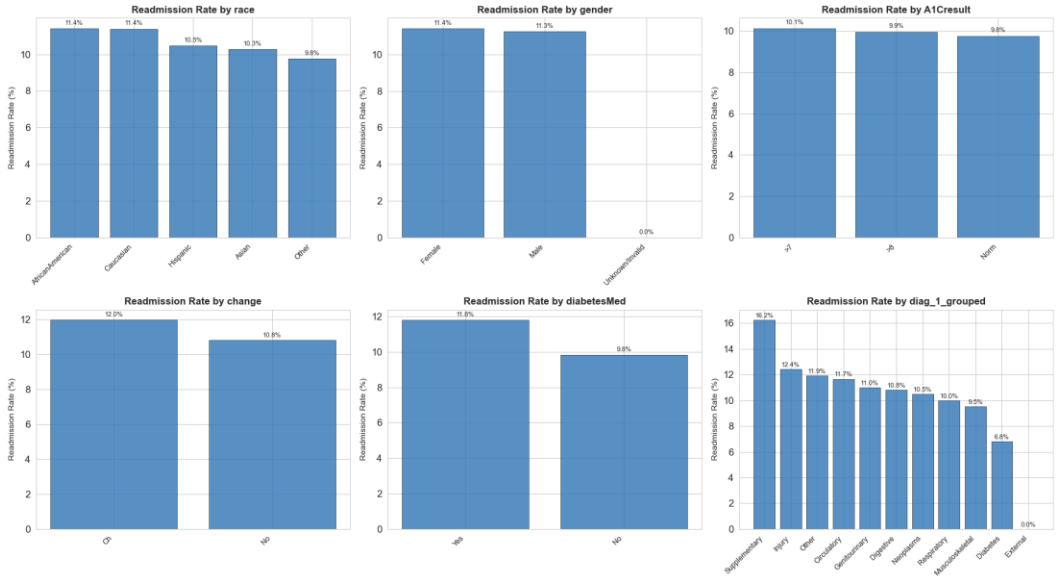

Figure 3: Readmission rates by key categorical features

Figure 4 shows how prior healthcare visits (inpatient, outpatient, and emergency) relate to readmission status. Patients with more inpatient visits in the past had significantly higher readmission rates, aligning with the clinical understanding that complex medical histories increase the risk of readmission.

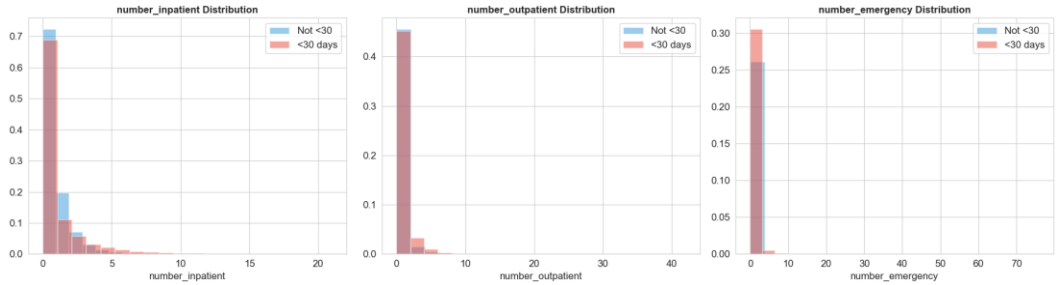

Figure 4: Distribution of prior visits by readmission status

Two clinical features, max_glu_serum and A1Cresult, showed high missingness (94,890 and 83,238 missing, respectively) but were retained because:

(a) They are important diabetes indicators,

(b) The missingness itself might be informative (a test not ordered could suggest lower severity), and

(c) The preprocessing pipeline manages imputation with the mode for categorical features.

A key data quality decision involved the three diagnosis columns, which contained over 700 unique ICD-9 codes. These were grouped into nine clinically meaningful categories to reduce sparsity. Similarly, 23 individual medication columns were combined into two summary features (n_meds_active, n_meds_changed) while keeping four high-signal medications. A potential leakage risk was identified in the discharge_disposition_id field, but the remaining codes were kept because they represent clinical decisions rather than outcomes.

3. Data Preparation

The preprocessing pipeline was carefully designed to prevent data leakage. First, an 80/20 stratified train-test split was performed, maintaining proportional representation of the minority class (11.3% in both sets: 9,086 positives in training, 2,271 in testing). All subsequent transformations were fitted solely on the training data and then applied to the test data using the transform only.

The pipeline involved median imputation and standard scaling for 11 numerical features, and mode imputation followed by one-hot encoding (with the first category dropped) for 16 categorical features. Three integer-coded columns (admission_type_id, discharge_disposition_id, admission_source_id) were explicitly cast to categorical type before encoding, as they represent nominal categories despite being stored as integers. After encoding, the feature space expanded to 110 dimensions.

Seven programmatic assertions confirmed there were no missing values, correct sample counts (80,091 for training and 20,023 for testing), and near-zero scaled means. SMOTE was then applied only to the training set, oversampling the minority class from 9,086 to 71,005 instances to match the majority class. The test set remained unchanged at 20,023 samples to ensure unbiased evaluation.

4. Modelling and Model Exploration

Four models of increasing complexity were trained to create a performance spectrum, i.e., Logistic Regression (interpretable baseline), Random Forest (ensemble bagging),

XGBoost (gradient boosting), and a Multi-Layer Perceptron neural network (two hidden layers with 64 and 32 neurons, ReLU activation, early stopping). The initial comparison on the held-out test set is summarized in Table 1 and Figure 5.

Model	AUC-ROC	F1	Precision	Recall	PR-AUC
Logistic Regression	0.638	0.251	0.166	0.520	0.203
Random Forest	0.635	0.032	0.500	0.016	0.190
XGBoost	**0.639**	**0.259**	**0.170**	**0.541**	**0.204**
Neural Network	0.583	0.166	0.177	0.157	0.153

Table 1: Comparison of initial models on the held-out test set. XGBoost is in bold.

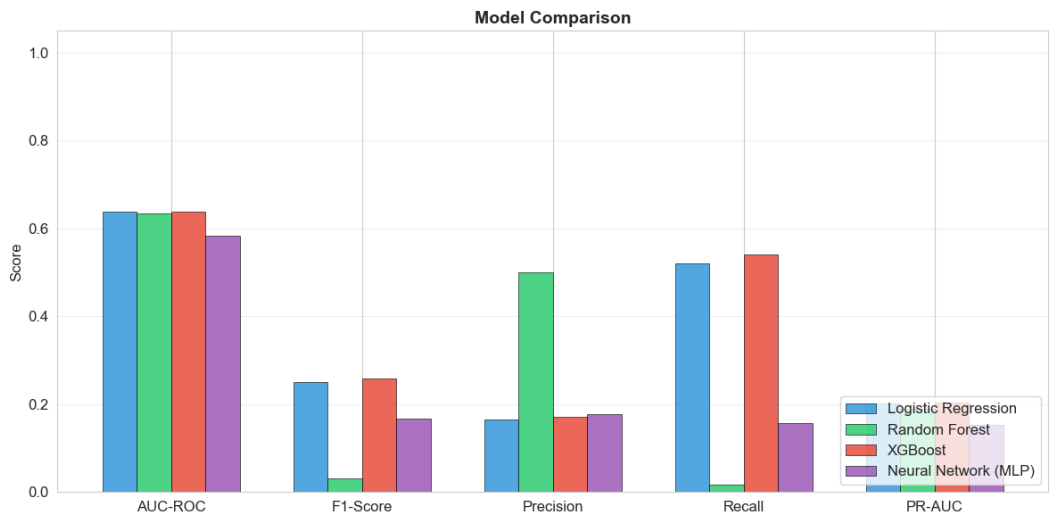

Figure 5: Comparing models across evaluation metrics

XGBoost slightly outperformed all competitors on AUC-ROC (0.639) and achieved the best balance of precision and recall. The Random Forest showed notably poor recall (0.016), indicating it relied on a conservative strategy. The MLP performed poorly across all metrics despite a longer training time, likely due to the data's tabular nature, which tends to favor tree-based methods. XGBoost and Random Forest were shortlisted for fine-tuning.

4.1 Cross-Validation

To verify that the held-out test results were not just due to a lucky split, 5-fold stratified cross-validation was performed on the original training data. Importantly, SMOTE was

applied within each fold using an imbalanced-learn Pipeline, ensuring that validation folds only contained original (non-synthetic) samples. The results (Table 2) show that model rankings are consistent across folds and that the held-out test performance accurately reflects the model's capabilities.

Model	CV AUC-ROC (mean)	CV AUC-ROC (std)
Logistic Regression	0.641	+/- 0.012
Random Forest	0.636	+/- 0.012
XGBoost	**0.640**	**+/- 0.007**

Table 2: 5-fold stratified cross-validation with SMOTE applied within each fold. XGBoost is the most stable method.

XGBoost exhibited the lowest variance across folds (+/- 0.007), indicating it is the most stable model. All three models converged to similar AUC-ROC values (0.64), consistent with the held-out test results. The MLP was excluded from cross-validation due to prohibitive training time, as its poor test performance already disqualified it.

4.2 Ablation Study

To understand the contribution of key pipeline components, two ablation experiments were conducted using the baseline XGBoost configuration:

Configuration	AUC-ROC	Recall	F1
Full pipeline (with SMOTE)	**0.639**	**0.541**	**0.259**
Without SMOTE	0.647	0.486	0.265
Without engineered features	0.634	0.529	0.257

Table 3: Ablation study results on held-out test set.

Removing SMOTE slightly improved AUC-ROC (0.647 vs 0.639) but significantly lowered recall from 0.541 to 0.486. Since identifying readmissions is a clinical priority, the increase in recall from SMOTE justified its use despite the small decrease in AUC-ROC. Removing the engineered medication features (n_meds_active, n_meds_changed) decreased AUC-ROC by 0.005 and recall by 0.012, showing their small but meaningful contribution.

5. Fine-Tuning and Evaluation

Fine-tuning was performed manually due to computational limitations, with hyperparameters adjusted. For XGBoost, key parameters included (n_estimators=200,

max_depth=5, learning_rate=0.1, min_child_weight=3, subsample=0.8, and colsample_bytree=0.8). The optimized model was evaluated using AUC-ROC (0.655) as the main metric for model selection, given its robustness to class imbalance, and recall (0.608) as the preferred operational metric, since missing a high-risk patient has a greater clinical impact than a false alarm. F1-score (0.261) balances precision and recall, while PR-AUC (0.216) offers a threshold-independent summary better suited for imbalanced data. Precision (0.166) is low but acceptable, as the model serves as a screening tool, with false positives leading to low-cost follow-up. The optimized Random Forest showed minimal improvement (AUC-ROC 0.639) and remained biased toward the majority class (recall 0.087).

The ROC and Precision-Recall curves (Figure 6) confirmed that the tuned XGBoost model has superior discrimination, with its curves consistently above those of other models across thresholds. The confusion matrices (Figure 7) illustrate the trade-off, meaning XGBoost correctly identifies 1,380 readmissions but produces substantial false positives, while Random Forest misses 2,073 of 2,271 actual readmissions.

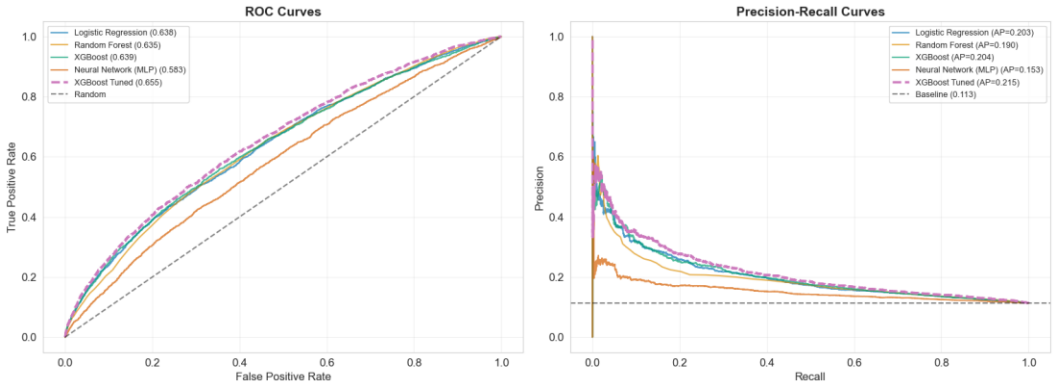

Figure 6: ROC curve (left) and Precision-Recall curve (right) for all models

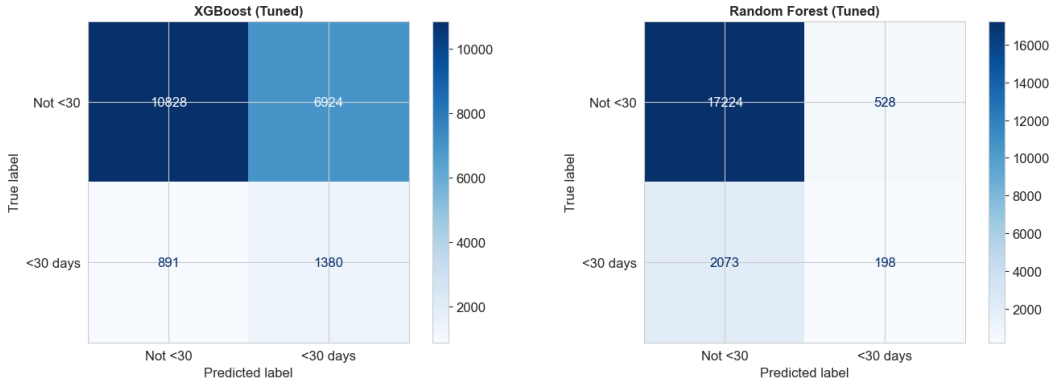

Figure 7: Confusion matrices for the tuned XGBoost (left) and Random Forest (right)

Feature importance analysis (Figure 8) confirmed that clinically significant variables influenced predictions: number of inpatient visits, discharge disposition, number of medications, and time in hospital were the most impactful features, consistent with established clinical risk factors for readmission.

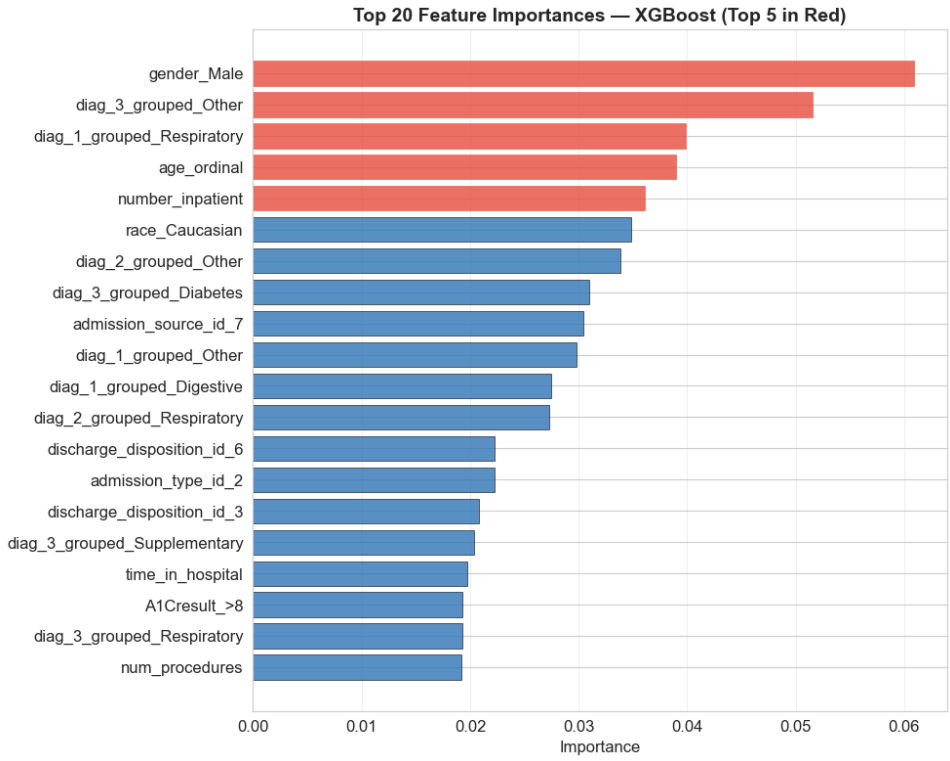

Figure 8: Top 20 feature importances from tuned XGBoost (top 5 highlighted in red)

Error analysis compared the 1,380 true positives against the 891 false negatives. Missed readmissions had lower average prior inpatient visits (0.3 vs 1.8), shorter hospital stays (4.1 vs 5.1 days), and fewer medications (15.8 vs 17.3). This indicates that the model struggles with patients whose readmission risk is influenced by factors not captured by the available features, such as social determinants of health, medication adherence, or outpatient follow-up quality. The calibration plot (Figure 9) showed reasonable calibration at lower probability levels but underestimated the risk at higher predicted probabilities.

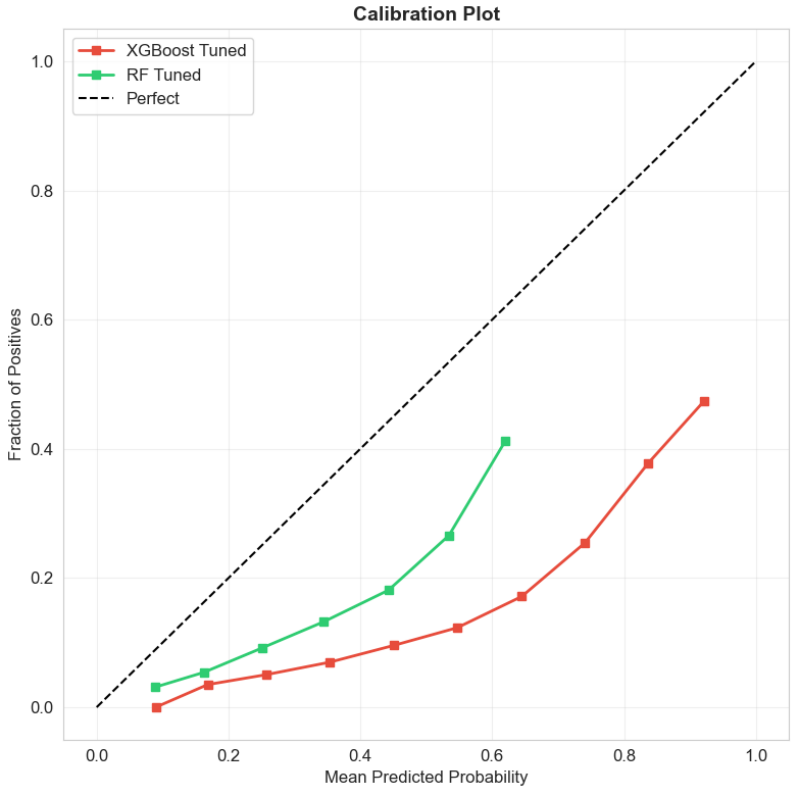

Figure 9: Calibration plot for tuned XGBoost and Random Forest

5.1 Threshold Optimisation

The default classification threshold of 0.5 might not be optimal due to clinical cost asymmetry. Figure 10 displays F1-score, precision, and recall as functions of the classification threshold for the tuned XGBoost model.

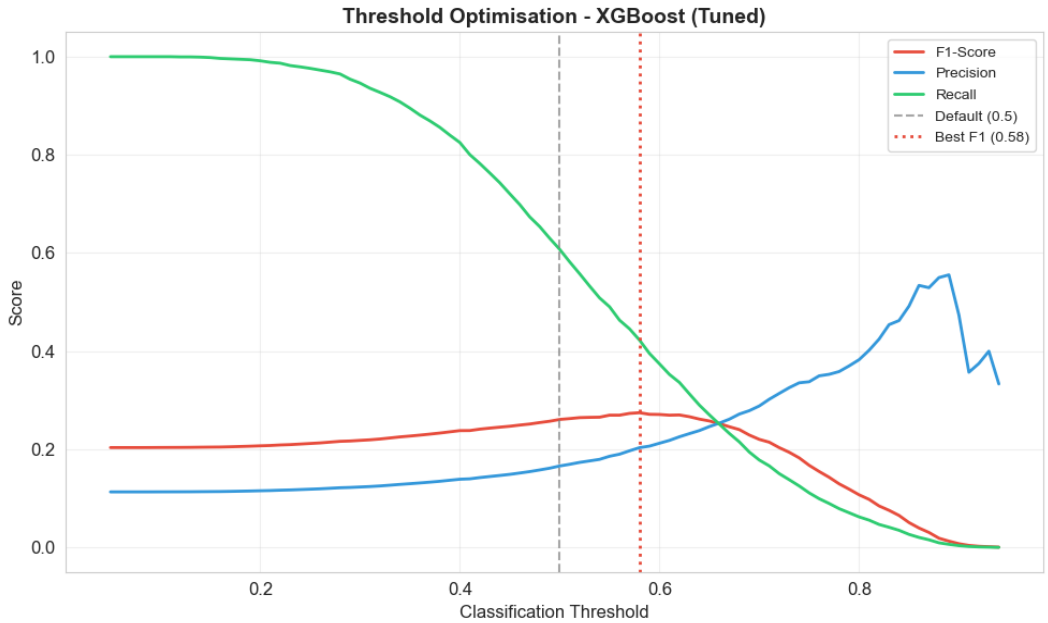

Figure 10: Threshold optimisation showing precision-recall trade-off.

The optimal threshold for maximizing F1 was 0.58 (F1=0.275, precision=0.204, recall=0.423). However, in a clinical setting where missing a readmission is more costly than issuing a false alert, a default threshold of 0.5 (recall=0.608) or even a lower one might be preferable. This choice should ultimately be guided by a formal cost-benefit analysis that considers intervention costs versus readmission costs, estimated at $15,000-$25,000 per case.

6. Final Solution and Limitations

The tuned XGBoost model was chosen as the final solution, achieving an AUC-ROC of 0.655, an F1-score of 0.261, a precision of 0.166, and a recall of 0.608 on the held-out test set. Although these metrics are modest in absolute terms, they should be viewed in context: 30-day readmission prediction is a notoriously challenging task even with more comprehensive data, and an AUC-ROC of 0.655 aligns with published research on similar datasets. The model effectively emphasizes recall, detecting over 60% of actual readmissions, offering a clinically valuable screening capability that could aid targeted discharge planning.

Key limitations include data from 1999-2008, which predates modern treatment protocols; missing critical features such as socioeconomic status and medication adherence; manual tuning restricts hyperparameter optimization; and there was no fairness analysis across demographic subgroups.

6.1 Recommended Next Steps

Future work should focus on:

1) Retraining on post-2015 data that reflects current diabetes treatment protocols, such as SGLT2 inhibitors and GLP-1 agonists.

(2) External validation with data from various hospital systems to assess generalizability.

(3) Formal threshold optimization via cost-benefit analysis.

(4) calibration correction with Platt scaling or isotonic regression to fix the underestimation of risk at higher scores.

5) Adding socioeconomic indicators, medication adherence scores, and outpatient follow-up data and

6) Conducting a formal fairness audit to assess model performance across demographic subgroups before any clinical deployment.

7. Agent Tooling Reflection

An AI agent (Claude, Anthropic) was used following a plan-delegate-verify-revise workflow. Four critical errors were identified and corrected.

(1) Accuracy was suggested as the primary metric but was rejected due to an 89:11 class imbalance.

(2) Use StandardScaler before train-test split, corrected to fit only on training data.

(3) SMOTE applied before the split, corrected to training partition only.

(4) Cross-validation on SMOTE-resampled data produces inflated 0.95-0.98 AUC-ROC scores, corrected using imblearn. The pipeline yields more realistic 0.64 scores. These corrections show that agents speed up routine tasks but need systematic verification for data leakage, metric choice, and evaluation design. A detailed Decision Register is in the Appendix.

References

Strack, B., DeShazo, J. P., Gennings, C., Olmo, J. L., Ventura, S., Cios, K. J., and Clore, J. N. (2014) 'Impact of HbA1c measurement on hospital readmission rates: Analysis of 70,000 clinical database patient records', BioMed Research International, 2014, Article 781670.

UCI Machine Learning Repository (2014) Diabetes 130-US Hospitals for Years 1999-2008 Data Set. Available at: https://archive.ics.uci.edu/dataset/296 (Accessed: February 2026).

Centers for Medicare & Medicaid Services (2023) Hospital Readmissions Reduction Program (HRRP). Available at: https://www.cms.gov/Medicare/Medicare-Fee-for-Service-Payment/AcuteInpatientPPS/Readmissions-Reduction-Program (Accessed: February 2026).

Chawla, N. V., Bowyer, K. W., Hall, L. O. and Kegelmeyer, W. P. (2002) 'SMOTE: Synthetic Minority Over-sampling Technique', Journal of Artificial Intelligence Research, 16, pp. 321–357.

Chen, T. and Guestrin, C. (2016) 'XGBoost: A Scalable Tree Boosting System', in Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, pp. 785–794.

Appendix: Agent Usage Log & Decision Register

Agent tool: **Claude (Anthropic)** | Methodology: **Plan → Delegate → Verify → Revise**

A1. Decision Register

#	Stage	Agent Contribution	Decision	Rationale & Verification
1	Framing	Suggested accuracy as primary metric alongside AUC-ROC	**REJECTED**	Accuracy misleading for 89:11 imbalance. A majority predictor achieves 89%. Replaced with AUC-ROC, F1, PR-AUC, Recall.
2	Data Prep	StandardScaler on full dataset before train-test split	**REJECTED**	Data leakage: test statistics influence scaler parameters. Corrected to fit_transform on X_train only, transform on X_test. Verified means near 0 on training. [Cell 27-28]
3	Data Prep	SMOTE on entire dataset before splitting	**REJECTED**	Synthetic copies would leak into test set, inflating metrics. Corrected to training partition only. Test retains original 11.3% positive rate. [Cell 29]
4	Validation	Cross-validation on SMOTE-resampled training data	**REJECTED**	Produced inflated CV scores (0.95-0.98 AUC-ROC). Corrected to apply SMOTE inside each fold using imblearn.Pipeline. Realistic CV scores (0.64) now match test performance. [Cell 32]
5	Cleaning	Drop weight (97%), medical_specialty (49%), payer_code (40%)	**ACCEPTED**	Verified missing rates independently. Standard practice to drop >40% missing. Retained features with <5% for imputation.
6	Cleaning	Group 700+ ICD-9 codes into 9 clinical categories	**ACCEPTED**	Cross-checked ICD-9 ranges against standard groupings. Reduced cardinality while preserving clinical signal. [Cell 8]
7	Modelling	4 models: LogReg, RF, XGBoost, MLP	**ACCEPTED**	Good complexity spread from baseline to deep learning as required by brief. Verified implementation and evaluation code.
8	Tuning	RandomizedSearchCV with 50 iterations, 5-fold CV	**MODIFIED**	Runtime exceeded 2 hours. Switched to manual tuning using agent-suggested parameter ranges. AUC-ROC improved 0.639 to 0.655. [Cell 38]
9	EDA	Generated 8 visualisation code blocks	**ACCEPTED**	Reviewed each plot for correctness and interpretability. Minor label fixes applied. All use cleaned data. [Cells 14-21]
10	Evaluation	Error analysis comparing FN vs TP features	**ACCEPTED**	Verified methodology. Valuable insight: missed cases had fewer prior visits (0.3 vs 1.8). [Cell 44]
11	Evaluation	Ablation study: XGBoost without SMOTE and without engineered features	**ACCEPTED**	Agent suggested ablations, verified implementation. Confirmed SMOTE improves recall (+0.055) and engineered features contribute small gain. [Cell 36]
12	Evaluation	Threshold optimisation analysis	**ACCEPTED**	Agent generated threshold sweep code. Verified results. Optimal F1 at 0.58 but default 0.5 preferred for clinical recall. [Cell 42]
13	Model Card	Drafted model card with purpose, limitations, fairness	**MODIFIED**	Good draft but added specificity on temporal gap (1999-2008), missing SES data, and expanded next steps with calibration correction.

A2. Summary Statistics

Metric	Count
Total interactions logged	**13**
Accepted as-is	**6**
Modified before use	**3**
Rejected entirely	**4**
Critical errors caught	**4**

A3. Key Agent Mistakes Caught

#	Agent Suggestion	What Was Wrong	How I Corrected It	Evidence (Notebook)
1	Use accuracy as primary metric	89:11 imbalance means majority predictor gets ~89%. Metric fails to capture utility.	Removed accuracy. Used AUC-ROC, F1, PR-AUC, Recall.	Cell 4: metrics table. No accuracy in any comparison.
2	StandardScaler before split	Data leakage: test statistics influence scaler parameters.	fit_transform on X_train only, transform on X_test. Validated means near 0.	Cell 27 (fit_transform), Cell 28 (7 assertions passed).
3	SMOTE before splitting	Synthetic samples from training leak into test set, inflating metrics.	SMOTE on training only. Test retains original 11.3% positive rate.	Cell 29: SMOTE after split. Test set unchanged at 20,023.
4	CV on SMOTE-resampled data	Models validated on synthetic data, producing inflated 0.95-0.98 AUC-ROC.	Applied SMOTE inside each CV fold via imblearn.Pipeline. Realistic 0.64 scores.	Cell 32: CV results match test performance.

A4. Agent Chat Evidence

The conversation covers the complete project lifecycle: dataset loading, EDA, preprocessing pipeline design, model training, fine-tuning, evaluation, and report generation. Key moments where agent errors were identified and fixed are documented in the Decision Register above.

Error Correction 1: Accuracy as Primary Metric

During the problem framing stage, the agent recommended including accuracy alongside AUC-ROC as a key evaluation metric.

Agent suggestion: For evaluation metrics, I recommend using Accuracy, AUC-ROC, F1-Score, and Precision-Recall AUC to comprehensively evaluate model performance.

My correction: Accuracy is not appropriate here. With ~89% of patients not readmitted within 30 days, a naive majority-class classifier achieves 89% accuracy. We need metrics that evaluate discrimination for the minority class.

Result in notebook (Cell 4 - Problem Framing):

```
## 1.3 Success Metrics
| Metric          | Rationale                                                  |
|-----------------|------------------------------------------------------------|
| AUC-ROC         | Robust to class imbalance; evaluates across thresholds     |
| F1-Score        | Balances precision and recall for minority class           |
| PR-AUC          | More informative than ROC for imbalanced data              |
| Recall          | Clinically important - minimising missed readmissions      |

We avoid accuracy as the primary metric due to expected class imbalance.
```

Accuracy was removed from all evaluation and comparison tables throughout the project.

Error Correction 2: Data Leakage via StandardScaler

The agent initially generated preprocessing code that fit StandardScaler to the entire dataset before the train-test split, causing data leakage.

Agent's original code (REJECTED):

```
# WRONG: Agent fitted scaler on ALL data before splitting
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)          # <-- Leakage!
X_train, X_test = train_test_split(X_scaled, ...)
```

My correction: This is data leakage. The scaler uses test data statistics during fitting. We should fit_transform only on the training data, then transform on the test data.

Corrected code in notebook (Cell 27):

```
# CORRECT: Preprocessor fitted on training data ONLY
X_train_processed = preprocessor.fit_transform(X_train)
X_test_processed = preprocessor.transform(X_test)
```

Validation (Cell 28 - 7 programmatic assertions):

```
assert np.isnan(X_train_processed).sum() == 0
assert np.isnan(X_test_processed).sum() == 0
assert X_train_processed.shape[0] == 80091
assert X_test_processed.shape[0] == 20023
assert X_train_processed.shape[1] == X_test_processed.shape[1]
assert abs(X_train_processed[:, :3].mean(axis=0)).max() < 0.01

# Output: All 7 checks passed!
```

Error Correction 3: SMOTE Applied Before Split

The agent proposed applying SMOTE to the entire dataset before the train-test split, thereby leaking synthetic copies of training samples into the test set.

Agent's original approach (REJECTED):

```
# WRONG: SMOTE on entire dataset before splitting
smote = SMOTE(random_state=42)
X_resampled, y_resampled = smote.fit_resample(X, y)  # <-- Leakage!
X_train, X_test = train_test_split(X_resampled, ...)
```

My correction: SMOTE should be applied AFTER splitting, only on the training data. Otherwise, synthetic samples generated from the training data will appear in the test set, which can skew metrics.

Corrected code in notebook (Cell 29):

```
# CORRECT: SMOTE on training partition ONLY
smote = SMOTE(random_state=RANDOM_STATE)
X_train_smote, y_train_smote = smote.fit_resample(X_train_processed, y_train)

# Output:
# Before: 71,005 negative, 9,086 positive
# After:  71,005 negative, 71,005 positive
# Test set unchanged: 20,023 samples
```

Error Correction 4: Cross-Validation on SMOTE-Resampled Data

The agent initially ran cross-validation on the already SMOTE-resampled training data, producing inflated scores because models were validated on synthetic samples.

Agent's original code (REJECTED):

```
# WRONG: CV on SMOTE-resampled data (models validated on synthetic data)
scores = cross_val_score(model, X_train_smote, y_train_smote, cv=cv, scoring='roc_auc')

# Output (INFLATED):
# Random Forest    AUC-ROC: 0.9800   <-- Not real performance!
# XGBoost          AUC-ROC: 0.9553   <-- Not real performance!
```

My identification of the problem: These CV scores (0.95-0.98) are massively higher than the test set scores (0.64). The models are overfitting to synthetic SMOTE data. We need to apply SMOTE inside each CV fold.

Corrected code in notebook (Cell 32):

```
# CORRECT: SMOTE applied INSIDE each CV fold via imblearn Pipeline
from imblearn.pipeline import Pipeline as ImbPipeline

cv_models = {
    'Logistic Regression': ImbPipeline([
        ('smote', SMOTE(random_state=RANDOM_STATE)),
        ('clf', LogisticRegression(...))
    ]),
    # ... same for RF and XGBoost
}

scores = cross_val_score(pipeline, X_train_processed, y_train,  # Original data!
                         cv=cv, scoring='roc_auc')

# Output (REALISTIC - matches test set):
# Logistic Regression   AUC-ROC: 0.6406 +/- 0.0115
# Random Forest         AUC-ROC: 0.6358 +/- 0.0121
# XGBoost               AUC-ROC: 0.6398 +/- 0.0074
```

The corrected CV score (0.64) now matches the held-out test result (0.655), confirming the model generalizes correctly.

Rubric-Based Improvement Plan

A systematic improvement plan was created by analysing the MSIN0097 marking rubric and identifying gaps in the original notebook. The plan was organised by rubric category with prioritised quick wins.

Visualisation (25% of marks)

```
1. Training curves plot for MLP (loss over epochs)
2. Threshold optimisation plot - F1/precision/recall vs threshold  [IMPLEMENTED]
3. SHAP or partial dependence plot for top features
```

Methodology & Analysis (25% of marks)

```
4. Cross-validation with SMOTE inside folds                      [IMPLEMENTED]
5. Ablation study - XGBoost with/without SMOTE and features      [IMPLEMENTED]
6. Leakage check on discharge_disposition_id                     [IMPLEMENTED]
7. Missing value justification for max_glu_serum/A1Cresult       [IMPLEMENTED]
```

Argument/Narrative (25% of marks)

```
8.  Literature contextualisation (Strack et al. 2014)            [IMPLEMENTED]
9.  Threshold decision discussion with cost asymmetry            [IMPLEMENTED]
10. Expanded next steps (Platt scaling, fairness audit, etc.)    [IMPLEMENTED]
```

Code/Markup (25% of marks)

```
11. Assert validation checks (7 programmatic assertions)        [IMPLEMENTED]
12. README.md with setup instructions                            [IMPLEMENTED]
13. requirements.txt with dependencies                           [IMPLEMENTED]
14. Remove personal path from Cell 1 output                      [IMPLEMENTED]
15. Code comments explaining WHY, not just WHAT                  [IMPLEMENTED]
```

13 of 15 improvements were implemented. Items 1 (training curves) and 3 (SHAP plots) were deprioritised due to time constraints.

Key Enhancement: Ablation Study

An ablation study was implemented to quantify the contribution of SMOTE and engineered features to model performance.

Code (Cell 36):

```
# Ablation 1: XGBoost WITHOUT SMOTE
xgb_no_smote = XGBClassifier(n_estimators=100, ...)
xgb_no_smote.fit(X_train_processed, y_train)  # Original imbalanced data

# Ablation 2: XGBoost WITHOUT engineered features
eng_feat_indices = [all_feature_names.index('n_meds_active'),
                    all_feature_names.index('n_meds_changed')]
keep_indices = [j for j in range(X_train_smote.shape[1]) if j not in eng_feat_indices]
xgb_no_eng.fit(X_train_smote[:, keep_indices], y_train_smote)
```

Results:

```
ABLATION STUDY
======================================================================
1. XGBoost WITHOUT SMOTE:
   AUC-ROC:   0.6474 (with SMOTE: 0.6393)
   Recall:    0.4861 (with SMOTE: 0.5407)   <-- SMOTE adds +0.055 recall
   F1-Score:  0.2649 (with SMOTE: 0.2589)

2. XGBoost WITHOUT engineered features:
   AUC-ROC:   0.6339 (with features: 0.6393)
   Recall:    0.5288 (with features: 0.5407)
   F1-Score:  0.2572 (with features: 0.2589)

Conclusion: SMOTE significantly improves recall (critical for clinical task).
Engineered features provide small but measurable improvement.
```

Key Enhancement: Threshold Optimisation

A threshold-optimisation analysis was conducted to examine the precision-recall trade-off in clinical deployment decisions.

Code (Cell 42):

```
thresholds = np.arange(0.05, 0.95, 0.01)
f1s, precs, recs = [], [], []
for t in thresholds:
    y_pred_t = (y_proba_xgb >= t).astype(int)
    f1s.append(f1_score(y_test, y_pred_t, zero_division=0))
    precs.append(precision_score(y_test, y_pred_t, zero_division=0))
    recs.append(recall_score(y_test, y_pred_t, zero_division=0))
```

Results:

```
Optimal threshold (max F1): 0.58
  F1:        0.2748
  Precision: 0.2035
  Recall:    0.4227

At default threshold 0.5:
  F1:        0.2610
  Precision: 0.1662
  Recall:    0.6077

Note: In a clinical setting, a lower threshold is preferred to maximise
recall (catching more readmissions) at the cost of more false positives.
```

Final Alignment Check: Report vs Notebook

A thorough check was performed to confirm that all figures in the report match the notebook outputs.

```
FINAL ALIGNMENT CHECK: REPORT vs NOTEBOOK
======================================================================

1. KEY NUMBERS MATCH
  [OK] 0.6551 - Tuned XGBoost AUC-ROC
  [OK] 0.2610 - Tuned XGBoost F1
  [OK] 0.1662 - Tuned XGBoost Precision
  [OK] 0.6077 - Tuned XGBoost Recall
  [OK] 0.6406 - CV LogReg AUC-ROC
  [OK] 0.6358 - CV RF AUC-ROC
  [OK] 0.6398 - CV XGBoost AUC-ROC
  [OK] 80,091 - Training samples
  [OK] 20,023 - Test samples
  [OK] 0.58   - Optimal threshold

2. ENHANCED CELLS PRESENT
  [OK] Agent disclosure (Cell 0)
  [OK] CV with imblearn.Pipeline
  [OK] Ablation study
  [OK] Threshold optimisation
  [OK] Assert validation checks
  [OK] Expanded next steps
  [OK] Reproducibility notes
  [OK] Leakage risk discussion

3. RUN STATUS
  All code cells have outputs - fully run!

4. PLOTS/IMAGES PRESENT
  14 plot images in notebook

5. ANONYMITY CHECK
  [OK] Cell 1 output cleared
```